

学号： 2106410113

大数据技术与基础 课后作业

学 生 姓 名： 张 艳 芳 专 业 班 级： 计 算 机 2101

第五章：抓取一个公开网站上的数据，数据必须是多页，存为 csv 格式；如果数据是多个 pdf，将其合并为一个 pdf。程序要尽量自动化，减少人为干预。

1、代码+注释

```
import requests
from lxml import etree
import csv

def getWeather(url):
    # 创建空列表用于存储天气信息
    weather_info = []
    # 设置请求头
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/92.0.4515.131
Safari/537.36 SLBrowser/8.0.0.12022 SLBChan/103'
    }
    # 发送 GET 请求获取网页内容
    resp = requests.get(url, headers=headers)
    # 使用 lxml 解析网页内容
    resp_html = etree.HTML(resp.text)
    # 使用 XPath 提取所需的天气信息
    resp_list = resp_html.xpath("//ul[@class='thru']//li")
    for li in resp_list:
        # 创建字典存储每日天气信息
        day_weather_info = {}
        # 提取日期时间信息
        day_weather_info['date_time'] = li.xpath("./div[1]/text()")[0].split(' ')[0]
        # 提取最高气温信息
        high = li.xpath("./div[2]/text()")[0]
        day_weather_info['high'] = high[:high.find('°C')]
        # 提取最低气温信息
        low = li.xpath("./div[3]/text()")[0]
        day_weather_info['low'] = low[:low.find('°C')]
        # 提取天气信息
        day_weather_info['weather'] = li.xpath("./div[4]/text()")[0]
        # 将每日天气信息添加到列表中
        weather_info.append(day_weather_info)
    return weather_info

# 存储所有月份的天气信息
weathers = []
# 遍历 1 到 12 月份
for month in range(1, 13):
```

```

# 构造 URL
weather_time = '2023' + ('0' + str(month) if month < 10 else
str(month))
url = f'https://lishi.tianqi.com/shenyang/{weather_time}.html'
# 获取该月份的天气信息
weather = getWeather(url)
# 将该月份的天气信息添加到列表中
weathers.append(weather)

# 将天气信息写入 CSV 文件
with open("weather.csv", "w", newline='') as csvfile:
    writer = csv.writer(csvfile)
    # 写入 CSV 文件的表头
    writer.writerow(["日期", "最高气温", "最低气温", '天气'])
    # 写入每日天气信息
    writer.writerows([list(day_weather_dict.values()) for
month_weather in weathers for day_weather_dict in month_weather])

```

2、程序说明

程序的主要功能是从指定的 URL 中获取沈阳历史天气信息，并将数据保存到 CSV 文件中。使用 requests 库发送 HTTP 请求，使用 lxml 库对 HTML 进行解析，并使用 csv 库将数据写入 CSV 文件。

3、运行截图

```

In [4]: runfile('D:/大数据python/天气.py', wdir='D:/大数据python')
202301
202302
202303
202304
202305
202306
202307
202308
202309
202310
202311
202312
[[{'date_time': '2023-01-01', 'high': '-3', 'low': '-18', 'weather': '晴'}, {'date_time':
'2023-01-02', 'high': '-4', 'low': '-18', 'weather': '晴'}, {'date_time': '2023-01-03',
'high': '-3', 'low': '-19', 'weather': '晴'}, {'date_time': '2023-01-04', 'high': '-1',
'low': '-15', 'weather': '晴'}, {'date_time': '2023-01-05', 'high': '3', 'low': '-8',
'weather': '晴'}, {'date_time': '2023-01-06', 'high': '-1', 'low': '-15', 'weather': '中
雪'}, {'date_time': '2023-01-07', 'high': '-2', 'low': '-13', 'weather': '雾'},
{'date_time': '2023-01-08', 'high': '5', 'low': '-13', 'weather': '多云'}, {'date_time':
'2023-01-09', 'high': '-1', 'low': '-13', 'weather': '晴'}, {'date_time': '2023-01-10',
'high': '6', 'low': '-3', 'weather': '晴'}, {'date_time': '2023-01-11', 'high': '7',
'low': '0', 'weather': '多云'}, {'date_time': '2023-01-12', 'high': '8', 'low': '0',
'weather': '阴'}, {'date_time': '2023-01-13', 'high': '4', 'low': '-14', 'weather': '雨夹
雪'}, {'date_time': '2023-01-14', 'high': '-11', 'low': '-19', 'weather': '阴'},
{'date_time': '2023-01-15', 'high': '-9', 'low': '-19', 'weather': '晴'}, {'date_time':
'2023-01-16', 'high': '-5', 'low': '-16', 'weather': '晴'}, {'date_time': '2023-01-17',

```

日期	最高气温	最低气温	天气
2023/1/1	-3	-13	晴
2023/1/2	-4	-18	晴
2023/1/3	-3	-19	晴
2023/1/4	-1	-15	晴
2023/1/5	3	-8	晴
2023/1/6	-1	-15	中雪
2023/1/7	-2	-13	雾
2023/1/8	5	-13	多云
2023/1/9	-1	-13	晴
2023/1/10	6	-3	晴
2023/1/11	7	0	多云
2023/1/12	8	0	阴
2023/1/13	4	-14	雨夹雪
2023/1/14	-11	-19	阴
2023/1/15	-9	-19	晴
2023/1/16	-5	-16	晴
2023/1/17	-2	-19	晴
2023/1/18	0	-8	晴
2023/1/19	2	-14	冻雨
2023/1/20	-9	-16	晴
2023/1/21	0	-13	多云
2023/1/22	-10	-20	小雪
2023/1/23	-17	-25	多云
2023/1/24	-15	-22	晴
2023/1/25	-4	-16	多云
2023/1/26	-8	-21	多云
2023/1/27	-7	-20	晴
2023/1/28	-2	-11	晴
2023/1/29	0	-15	多云

第六章：葡萄酒数据集（wine.data）搜集了法国不同产区葡萄酒的化学指标。试建立决策树、随机森林和 xgBoost3 种分类器模型，比较各种分类器在此数据集上的效果。

【提示】：每种分类器，需要对参数进行尝试，找出此种分类算法的较优模型，再与其他分类器性能进行比较。

1、代码+注释

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

# 读取数据集
data = pd.read_csv('wine.data', header=None)

# 提取特征矩阵 X 和目标向量 y
X = data.iloc[:, 1:]
y = data.iloc[:, 0]

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

# 将目标向量 y 进行标签编码
xgy = le.fit_transform(y)

# 将数据集分割为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

```

# 再次将数据集分割为训练集和测试集，这次使用标签编码后的目标向量 xgy
X_train, X_test, xgy_train, xgy_test = train_test_split(X, xgy,
test_size=0.3, random_state=42)

# 创建决策树分类器对象并进行训练
dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)
dtc_score = dtc.score(X_test, y_test)

# 创建随机森林分类器对象并进行训练
rfc = RandomForestClassifier(random_state=42)
rfc.fit(X_train, y_train)
rfc_score = rfc.score(X_test, y_test)

# 创建 XGBoost 分类器对象并进行训练
xgb = XGBClassifier(random_state=42)
xgb.fit(X_train, xgy_train)
xgb_score = xgb.score(X_test, xgy_test)

# 打印输出决策树分类器、随机森林分类器和 XGBoost 分类器在测试集上的准确率
print('Decision Tree Classifier score:', dtc_score)
print('Random Forest Classifier score:', rfc_score)
print('XGBoost Classifier score:', xgb_score)

```

2、程序说明

程序使用了 Pandas、Scikit-learn 和 XGBoost 库来训练和评估分类模型，用测试集测试每个模型的准确度，比较得出最合适的模型。

3、运行截图

```

Decision Tree Classifier score: 0.9629629629629629
Random Forest Classifier score: 1.0
XGBoost Classifier score: 0.9444444444444444

```